Note 1

Introduction to Data Warehousing

- The data warehousing industry has matured significantly since the first edition of *The Data Warehouse Toolkit* was published in 1996.
- Organizations of all sizes have embraced data warehousing, resulting in thousands of constructed warehouses.
- The volume of data continues to grow, requiring effective tools for design, development, and usage.
- Dimensional modeling is recognized as the dominant technique for data warehouse presentation, emphasizing **simplicity** for user understandability and query performance.
- This edition aims to compile collective thoughts on dimensional modeling and share practical design recommendations based on real-world scenarios.

Note 2

Intended Audience

- The book targets data warehouse designers, implementers, managers, and business analysts.
- It is crucial for all project team members to understand dimensional modeling due to its impact on various aspects of warehouse implementation.
- The content is designed to be accessible to a broad audience, regardless of their primary responsibilities.

Note 3

Chapter Overview

- **Chapter 1:** Introduces dimensional modeling, components of data warehouse architecture, and core vocabulary.
- **Chapter 2:** Focuses on retail sales as an example, discussing a four-step design process and essential dimension tables.
- **Chapter 3:** Discusses inventory within the retail context, emphasizing data warehouse bus architecture and conformed dimensions.
- **Chapter 4:** Explores procurement and the importance of understanding the value chain in data warehousing.
- **Chapter 5:** Covers order management, highlighting core metrics and challenges in modeling order information, such as multiple currencies and junk dimensions.

Note 4

Key Concepts in Dimensional Modeling

- Dimensional modeling is essential for **user understandability** and **query performance**.

- Common concepts include:
- Slowly Changing Dimensions (SCD)
- Factless Fact Tables
- Heterogeneous Products
- The book provides a toolkit of dimensional design principles suitable for both beginners and advanced practitioners.

Note 5

Design Techniques and Case Studies

- The book is organized around business vignettes to illustrate design techniques through practical examples.
- Each chapter builds on the previous one, introducing more advanced content progressively.
- Familiarity with earlier chapters is crucial for understanding later, more complex topics.

Note 6

Introduction to Data Warehousing

- The chapters ahead illustrate various **modeling techniques** layered on each other, akin to **overlaying transparencies**.
- A high-level overview of **data warehouse project lifecycles** will be provided, referencing *The Data Warehouse Lifecycle Toolkit*.
- The final chapter discusses **future trends** in data warehousing, offering a preview of anticipated developments.
- A detailed **glossary** will serve as a reference resource, bridging general business understanding with case studies.
- The book's **companion website** offers resources such as design tips, articles, and information about Kimball University classes.

Note 7

Summary of Key Goals

- The primary aim of the book is to communicate **standard techniques** for dimensional data warehouse design, emphasizing a **business user-driven** approach.
- Readers should understand that a data warehouse must be created from a **simple dimensional perspective**.
- The book will begin with a **dimensional modeling primer** to establish common terminology and architectural concepts.
- Techniques will be discussed, starting with examples from the **retail industry**.
- An understanding of the **business context** is crucial for successful data warehousing.

Note 8

Dimensional Modeling Primer

- The first chapter establishes the **foundations for case studies**, focusing on the macro perspective of data warehousing.
- Emphasis is placed on the importance of addressing **business needs** first, rather than merely utilizing tools and techniques.
- Key concepts discussed include:
- **Business-driven goals** of a data warehouse.
- The role of **normalized models** within the warehouse.
- Fundamental **vocabulary for dimensional modeling**.
- Common **pitfalls in data warehousing** to avoid.
- The chapter aims to blend the skills of a **DBA** with those of a **business analyst**.

Note 9

Different Information Worlds

- Organizations manage information in two forms: **operational systems** (where data is input) and **data warehouses** (where data is analyzed).
- **Operational system users** are focused on individual records and routine tasks, while **data warehouse users** analyze trends and patterns across multiple records.
- There is a critical distinction between operational processing and data warehousing needs, which are often misunderstood.
- Many so-called data warehouses are simply copies of operational systems, failing to meet the unique needs of data warehouse users.
- Business users require accessible, understandable, and consistent data for effective decision-making.

Note 10

Goals of a Data Warehouse

- Key goals derived from business management feedback include:
- Easy **access** to a wealth of data.
- Ability to **slice and dice** data effectively.
- Tools must be user-friendly, allowing for **direct business access** to data.
- Consistent presentation of **business metrics** to avoid discrepancies.
- Support for **fact-based decision-making**.
- Data warehouses must ensure:
- Information is **credible** and thoroughly cleansed.
- Adaptability to **change** in user needs and technology.
- Security of **confidential information**.
- Foundation for improved **decision-making** and acceptance by business users.

Note 11

The Publishing Metaphor for Data Warehousing

- The role of a data warehouse manager is likened to that of a **publishing editor-in-chief**, focusing on serving the needs of users rather than just managing technology.
- Responsibilities include:
- Understanding user demographics and needs.
- Identifying effective users and potential new users.
- Selecting actionable data for presentation.
- Ensuring data quality and consistency.
- Continuously adapting to changing data requirements.
- This metaphor emphasizes the importance of focusing on users and their needs to ensure the success of the data warehouse.

Note 12

Components of a Data Warehouse

- A complete data warehouse environment consists of four main components:

1. **Operational Source Systems**:
- Capture transactions; primarily focused on performance.
- Maintain limited historical data.

1. **Data Staging Area**:
- Handles the extract-transform-load (ETL) process.
- Prepares raw data for user access and query.

1. **Data Presentation Area**:
- The interface where users access and query the data.

1. **Data Access Tools**:
- Tools like ad hoc queries and report writers that facilitate user interaction with the warehouse.
- Each component serves a unique function, contributing to the overall effectiveness of the data warehouse.

Note 13

Data Staging Area

- The **data staging area** primarily involves **sorting** and **sequential processing**.
- Often not based on relational technology, it may consist of **flat files**.
- After validating data against **business rules**, creating a full **third-normal-form (3NF)** database may be unnecessary.
- If data arrives in a 3NF format, cleansing tasks can utilize **normalized structures**.
- However, this leads to **double processing**, requiring more resources and time.
- A **normalized database** is acceptable for staging but must be off-limits for user queries to maintain **performance** and **understandability**.
- The ETL process culminates in loading data into **dimensional tables** for each data mart, which must then index this data for **query performance**.
- Publication of new data must communicate changes in dimensions and assumptions to the user community.

Note 14

Data Presentation Area

- The **data presentation area** is where data is organized for direct user access via **querying tools**.
- This area is what the business community interacts with, making it the **data warehouse** from their perspective.
- Typically referred to as a series of **integrated data marts**, each data mart corresponds to a single business process.
- Data must be presented in **dimensional schemas**, as this method has been accepted as the best practice for data warehouse users.
- **Dimensional modeling** simplifies data representation, allowing users to visualize data intuitively, thus enhancing usability.
- Complicated models, like **3NF**, make querying challenging and could lead to poor performance.
- Dimensional models contain detailed **atomic data** necessary for ad hoc queries, while summary data enhances performance but should not replace it.

Note 15

Dimensional Modeling vs. Normalized Modeling

- **Dimensional modeling** differs from **3NF modeling**, focusing on user understandability and high-performance retrieval.
- 3NF aims to eliminate data redundancy, leading to complex structures that are difficult for users to navigate.
- While 3NF models help operational processes, they are inadequate for data warehouse queries.
- **Symptoms of overly complex schemas** include:
- High costs with minimal query performance.
- Dependence on IT for query writing.
- Simple queries requiring extensive SQL coding.
- Unhappy business users unable to access the system directly.
- Dimensional models package data for intuitive access, enhancing user experience and query performance.

Note 16

Data Mart Architecture and Integration

- All data marts should be built using **common dimensions and facts**, referred to as **conformed dimensions**.
- This is crucial for integrating various data marts, avoiding **isolated stovepipe applications**.
- A robust data warehouse requires commitment to a **bus architecture**, which facilitates decentralized development of the enterprise data warehouse.
- The presentation area data must be **dimensional**, **atomic**, and conform to the bus architecture.
- Data marts can utilize **star schemas** or OLAP cubes, depending on the platform.
- Dimensional modeling is adaptable to both relational and multidimensional databases, aiming for consistent logical designs.

Note 17

Data Access Tools and Metadata

- **Data access tools** range from simple ad hoc queries to complex data mining applications, primarily querying the presentation area.
- Most users will benefit from **prebuilt analytic applications**, with a small percentage using ad hoc tools.
- **Metadata** encompasses all non-data information in the data warehouse, vital for operational and technical documentation.
- Different forms of metadata support various needs, including operational source system metadata and presentation area documentation.
- An effective metadata framework is crucial for cataloging and integrating information, paralleling library resources.
- An **Operational Data Store (ODS)** may serve specific needs, such as operational reporting or real-time interactions, but should not be a separate entity if the warehouse can meet requirements.

Note 18

Fact Tables

- A **fact** represents a **business measure**. For example, in a marketplace, you can track quantities sold and dollar sales amounts per product per store.
- Each row in a fact table corresponds to a **measurement**; all measurements must be at the same **grain**.
- Useful facts are typically **numeric** and **additive** (e.g., dollar sales amount).
- **Additivity** is crucial for data warehouse applications, as they often retrieve numerous rows at once, requiring aggregation.
- Not all facts are fully additive:
- **Semi-additive facts** can only be added along some dimensions.
- **Nonadditive facts** cannot be summed and may require counts or averages instead.
- Fact tables are generally **sparse**; rows without sales activity are omitted.
- Fact tables typically consume **90%** of the space in a dimensional database, being deep in rows but narrow in columns.
- Fact table grains can be categorized into three types: **transaction**, **periodic snapshot**, and **accumulating snapshot**.
- Fact tables contain **foreign keys** that connect to dimension tables, ensuring referential integrity.

Source Pages: [35, 36, 37]

Note 19

Dimension Tables

- **Dimension tables** complement fact tables, containing **textual descriptors** of the business.

- A well-designed dimension table may contain **50 to 100 attributes**.
- Dimension attributes serve as the primary source for **query constraints**, **groupings**, and **report labels**.
- Attributes should be **textual** and **discrete**, avoiding cryptic abbreviations.
- Dimension tables often represent **hierarchical relationships** (e.g., products categorized by brand and category).
- The goal is to minimize reliance on codes; instead, use explicit dimension attributes for clarity.
- Dimension tables are typically **highly denormalized** to aid usability and performance, often being less than **10%** of total data storage requirements.
- Designers may face uncertainty in classifying a numeric field as either a fact or dimension attribute, often resolved by examining its role in calculations or constraints.

Source Pages: [36, 38, 39]

Note 20

Bringing Together Facts and Dimensions

- Fact tables and dimension tables are integrated into a **dimensional model**, often depicted as a **star join schema**.
- The star schema offers simplicity and symmetry, making it easier for business users to understand.
- Performance benefits arise from the simplicity of the model; fewer joins lead to more efficient processing.
- Dimensional models are **extensible**, allowing for the addition of dimensions and facts without reloading data.
- The relationship between fact and dimension tables reflects in reporting, where dimension attributes provide labels and fact tables supply numeric values.
- Converting existing normalized ER diagrams into dimensional models involves separating business processes and identifying numeric, additive facts.

Source Pages: [40, 41, 42]

Note 21

Dimensional Modeling Myths

- **Myth 1**: Dimensional models are for summary data only.
- Reality: They should provide access to the most detailed data for flexibility in querying.
- **Myth 2**: Dimensional models are departmental, not enterprise solutions.
- Reality: They should be organized around **business processes**, not departments.
- **Myth 3**: Dimensional models are not scalable.
- Reality: Modern fact tables can handle billions of rows and are scalable across industries.
- **Myth 4**: Dimensional models require predictable usage patterns.
- Reality: They are designed to adapt to changing business needs, making them responsive to new queries.

Source Pages: [43]

Note 22

Dimensional Modeling Flexibility

- The **flexibility** of dimensional structures in data marts is crucial for effective querying.
- **Fact tables** should be built at the **most granular level** to avoid issues with prematurely aggregated data.
- Users encounter **analytic challenges** when attempting to drill down into non-existent details in summary tables.
- Developers face difficulties integrating new dimensions, attributes, or facts due to these **premature summaries**.
- Starting with the **lowest detail** provides maximum flexibility and extensibility for future changes.

Note 23

Myth 5: Integration of Dimensional Models

- Dimensional models and data marts can be integrated if they follow the **data warehouse bus architecture**.
- **Standalone solutions** arise from databases that do not conform to this architecture.
- Issues with integration should not be blamed on dimensional modeling itself; it's the **lack of adherence** to fundamental principles that leads to stovepipe solutions.

Note 24

Common Pitfalls in Data Warehousing

- **Pitfall 10:** Focusing too much on **technology** over business requirements.
- **Pitfall 9:** Not securing a strong **management sponsor** for the data warehouse initiative.
- **Pitfall 8:** Overcommitting to a **large, multiyear project** instead of manageable iterations.
- **Pitfall 7:** Spending resources on a **normalized data structure** while neglecting a dimensional presentation area.
- **Pitfall 6:** Prioritizing **operational performance** over user-friendly query performance.

Note 25

Continuing Common Pitfalls

- **Pitfall 5:** Overcomplicating the **presentation area** data, making it difficult for users.
- **Pitfall 4:** Building dimensional models **in isolation**, disregarding a cohesive data architecture.
- **Pitfall 3:** Loading only **summarized data** into the presentation area, limiting analytical capabilities.
- **Pitfall 2:** Assuming that business needs and data are **static**.

- **Pitfall 1:** Ignoring that **user acceptance** is critical for data warehouse success; without it, efforts may be futile.

Note 26

Dimensional Modeling Overview

- Important concepts include:
- A **four-step process** for designing dimensional models.
- Understanding **transaction-level fact tables** and types of facts (additive vs non-additive).
- Identifying sample **dimension table attributes** and causal dimensions, like promotions.
- Recognizing **degenerate dimensions** such as transaction ticket numbers and the importance of surrogate keys.

Note 27

Four-Step Dimensional Design Process

1. **Select the business process** to model (e.g., purchasing, orders, shipments).
1. **Declare the grain** of the business process, specifying what each fact table row represents.
- Example grains:
- Individual line item on a sales ticket.
- Daily inventory levels.
1. **Choose the dimensions** applicable to each fact table row, identifying attributes that describe the data.
1. **Identify the numeric facts** that will populate each fact table row, ensuring they align with the declared grain.

Note 28

Case Study: Retail Sales

- The retail business case study focuses on a **grocery chain** with 100 stores and ~60,000 SKUs.
- Data collection occurs at **cash registers** (POS system) and vendor deliveries.
- Management's key decisions revolve around **pricing**, **promotions**, and maximizing profits.
- The business process modeled will be **POS retail sales**, analyzing product sales across various conditions.

Note 29

Grain Declaration in Retail Case Study

- The grain for the retail sales model will be the **individual line item** on a POS transaction.
- This atomic data provides maximum dimensionality and analytic flexibility.
- Users may want to analyze trends, promotions, or sales differences using detailed data.

- Avoiding summarized data ensures users can drill down into specifics without encountering analytic walls.

Note 30

Choosing Dimensions and Facts

- After declaring the grain, dimensions such as **date**, **product**, **store**, and **promotion** will be selected.
- The POS transaction ticket number will also be included as a special dimension.
- Facts to be included in the fact table are:
- **Sales quantity**
- **Sales dollar amount**
- **Cost dollar amount**
- All facts must align with the declared grain to ensure valid and meaningful analysis.

Note 31

Gross Profit Calculation

- **Gross Profit Formula**: Gross Profit is calculated by subtracting the cost dollar amount from the sales dollar amount (revenue).
- **Additivity**: Gross profit is additive across all dimensions, allowing calculations for any combination of products, stores, and dates.
- **Storage Recommendation**: It is generally recommended to store calculated facts like gross profit in the database to avoid user error and ensure consistency across reporting applications.
- **Alternative Approaches**: Some suggest calculating gross profit in a view or within query tools. This may minimize user error but requires strict access controls to avoid direct interaction with the underlying table.
- **Gross Margin**: Gross Margin is calculated as Gross Profit divided by revenue and is a non-additive fact. It cannot be summarized across dimensions.
- **Design Principle**: Percentages and ratios should be computed from sums of their components rather than individual ratios.

Note 32

Date Dimension Overview

- **Importance**: The date dimension is vital in every data mart as it serves as a time series.
- **Pre-Building**: The date dimension can be built in advance, typically covering 5 to 10 years of data, resulting in around 3,650 rows.
- **Attributes**: Essential attributes for the date dimension include:
- **Day of Week**
- **Month Number**
- **Fiscal Indicators**
- **Holiday Indicator** (e.g., Holiday/Nonholiday)
- **Weekday Indicator** (e.g., Weekday/Weekend)

- **SQL Date Limitations**: SQL date functions are limited in usability, as they do not support filtering by non-standard attributes like holidays or fiscal periods, emphasizing the need for a dedicated date dimension table.
- **Design Principle**: Calendar logic should reside in dimension tables rather than in application code, improving usability and maintaining efficiency in queries.

Note 33

Product Dimension Details

- **SKU Count**: The product dimension may contain up to 150,000 or more SKUs, depending on merchandising schemes and historical product data.
- **Sourcing**: This dimension is typically sourced from the operational product master file, updated regularly from headquarters.
- **Descriptive Attributes**: The product dimension includes a variety of descriptive attributes, such as:
- **SKU Description**
- **Brand Description**
- **Category and Department Descriptions**
- **Fat Content, Diet Type, Package Type, etc.**
- **Merchandise Hierarchy**: Products are organized into a hierarchy (SKU -> Brand -> Category -> Department) which supports analysis across different levels.
- **Drilling Capability**: Users can drill down or up through the hierarchy or by any attribute, allowing versatile analysis of sales data.

Note 34

Store Dimension

- The **store dimension** details every store in a grocery chain, unlike the product master file which is universally available.
- Store master files may not be comprehensive, requiring IT to compile information from multiple operational sources.
- Stores can be categorized by **geographic attributes** such as ZIP code, county, or state, allowing for hierarchies like store districts and regions.
- Multiple hierarchies can be represented within a single dimension table, ensuring unique attribute names and values across these hierarchies.
- Essential attributes for the store dimension include:
- Store Key (PK)
- Store Name
- Store Number (Natural Key)
- Store Address (Street, City, County, State, Zip)
- Store Manager
- Selling and Total Square Footage
- First Open Date and Last Remodel Date.

Note 35

Promotion Dimension

- The **promotion dimension** outlines conditions under which products are sold, including price reductions and advertising.
- This dimension is deemed a **causal dimension**, as it helps in analyzing sales changes due to promotional activities.
- Key considerations for evaluating promotional effectiveness include:
- **Sales Lift**: Increase in sales during promotions versus baseline.
- **Time Shifting**: Sales may drop before or after promotions, affecting overall metrics.
- **Cannibalization**: Sales shifts from regular products to those on promotion.
- **Market Growth**: Overall sales growth in the category during promotions.
- **Profitability**: Assessing profit adjustments considering promotional costs.
- A recommended promotion dimension table includes:
- Promotion Key (PK)
- Promotion Name
- Price Reduction Type
- Promotion Media Type
- Promotion Cost
- A special row for "**No Promotion in Effect**" is crucial to avoid null keys in the fact table, maintaining referential integrity.

Note 36

Promotion Coverage Factless Fact Table

- The **promotion coverage factless fact table** addresses the gap of tracking products that were on promotion but did not sell.
- This table captures relationships without measurement metrics, enabling analysis of promotional effectiveness.
- Key attributes include:
- Date
- Product
- Store
- Promotion
- The grain differs from the sales fact table, as it records every product under promotion daily, regardless of sales.
- Analyzing products that did not sell involves querying both the promotion coverage and sales fact tables, allowing for set difference operations.

Note 37

Degenerate Transaction Number Dimension

- The **POS transaction number** serves as a degenerate dimension in the retail sales fact table, representing unique transactions.
- It does not join with a dimension table, as all relevant descriptive data has been extracted into other dimensions.
- Common attributes for degenerate dimensions include order numbers and invoice numbers, typically appearing in fact tables representing single transactions.
- If attributes remain after dimension creation, a normal dimension record may be added

instead of keeping it as degenerate.

Note 38

Retail Schema in Action

- A business user might query **weekly sales by promotion** for a specific product category, revealing relationships between sales and promotional activities.
- Example query results show sales volumes categorized by week and promotion:
- January 6, 2002: No Promotion - $22,647
- January 20, 2002: Super Bowl Promotion - $7,248
- January 27, 2002: Super Bowl Promotion - $13,798
- This illustrates the importance of high-quality dimension attributes for effective querying and reporting.

Note 39

Retail Sales Schema Extensibility

- The retail schema can be extended to incorporate a **frequent shopper program**, allowing for detailed shopper behavior analysis.
- A frequent shopper dimension table would be created with a corresponding key in the fact table to track individual purchases.
- Historical transactions would use a placeholder key like "**Prior to Frequent Shopper Program**" to avoid null entries.
- Additional dimensions can include time of day and clerk information, enhancing data granularity without disrupting existing applications.
- The design's flexibility allows for seamless incorporation of new dimensions without compromising the integrity of preexisting data structures.

Note 40

New Dimension Attributes

- New textual descriptors can be added as new columns in a dimension.
- Existing applications remain unaffected by these new attributes.
- If the attributes are only available after a certain date, older records should be marked as "Not Available".
- New dimensions can be integrated by adding a foreign key to an existing fact table and populating it with the primary key of the new dimension.
- New measured facts can be easily added to the fact table if they are available at the same grain as existing facts; if not, a new fact table may need to be created.

Note 41

Dimension Normalization (Snowflaking)

- Normalization may make some modelers uncomfortable as it involves separating attributes into different tables.
- Example: Instead of storing a department description in the product table, a modeler might prefer to store a department code and separate department dimension.
- Snowflaking can complicate the schema and reduce performance because of increased joins and complexity.
- Benefits of denormalized schemas include:
- Easier for users to browse dimensions.
- Better performance due to less complex queries.
- Enhanced use of bitmap indexes for low-cardinality fields.
- Snowflaking is acceptable in certain situations, but should be used conservatively.

Note 42

Too Many Dimensions in Fact Table

- Designing a fact table with too many dimensions can lead to performance issues.
- Best practice: Limit dimensions in a fact table to fewer than 15.
- Correlated dimensions should be combined to simplify the model.
- Example of a 'centipede' design: A fact table with excessive foreign keys leading to numerous dimension tables, complicating both usability and performance.
- Combining dimensions that are statistically correlated can lead to a more efficient schema.

Note 43

Surrogate Keys

- Surrogate keys are recommended over operational production codes for better data integrity and performance.
- These keys are sequential integers assigned to dimension records, simplifying joins to fact tables.
- Advantages of surrogate keys include:
- They buffer the data warehouse from operational changes and key overlap problems.
- They are smaller than operational codes, leading to smaller fact tables and better performance.
- Surrogate keys help manage situations with no operational codes, like "No Promotion in Effect".
- Using surrogate keys for date dimensions allows for sequential ordering and efficient partitioning of fact tables.

Note 44

Retail Sales Schema

- The retail sales schema comprises **three additional dimensions**: shopper, clerk, and time of day.
- Each shopper and clerk is allocated **4 bytes**, while the time of day is allocated **2 bytes**.

- The total width of the fact table is **48 bytes**, leading to a total size of **48 GB** for a billion-row fact table.
- The schema provides detailed insights into what products are purchased under various conditions, but it lacks straightforward capabilities for analyzing product combinations sold together.
- To analyze product combinations, **market basket analysis** is used, which reveals insights into product merchandising by looking at combinations that sell together.

Note 45

GMROI Calculation

- The **Gross Margin Return on Investment (GMROI)** is calculated using the formula:

$$ \text{GMROI} = \frac{\text{Total Quantity Sold} \times (\text{Value at Latest Selling Price} - \text{Value at Cost})}{\text{Daily Average Quantity on Hand} \times \text{Value at Latest Selling Price}} $$

- **Key Components of GMROI:**
- **Total Quantity Sold**: Number of items sold during a period.
- **Value at Latest Selling Price**: Current selling price of inventory.
- **Value at Cost**: Original cost of inventory.
- A **high GMROI** indicates effective inventory management with quick sales and good profit margins, while a **low GMROI** suggests poor performance.
- GMROI is not stored in the fact table as it is not additive. It can be calculated from constituent columns across fact rows.
- Consider including quantity sold, value at cost, and value at the latest selling price in an inventory snapshot fact table for better analysis.

Note 46

Inventory Transactions

- **Inventory Transactions** at warehouses include:
- Receive product
- Place product into inspection hold
- Return product to vendor due to inspection failure
- Authorize product for sale
- Ship product to customer
- Each transaction records key details: date, product, warehouse, vendor, transaction type, and inventory quantity impact.
- Transaction-level fact tables provide detailed insights but are impractical for broad analysis due to their complexity.
- While transaction tables track specific actions, a snapshot table gives a cumulative view of inventory processes.

Note 47

Accumulating Snapshot Model

- An **accumulating snapshot** tracks a shipment's journey through various milestones in a warehouse.
- Example milestones: Receiving, Inspection, Bin Placement, Authorization to Sell, Picking, and Shipping.
- This model allows tracking the status of a product until it leaves the warehouse, focusing on discrete events and updates.
- The accumulating snapshot fact table has multiple date-valued foreign keys, representing different stages in the process.
- It is not commonly used for long-running processes but is beneficial for detailed tracking of product shipments.

Note 48

Value Chain Integration

- **Integration across the value chain** is crucial for evaluating performance and understanding customer relationships.
- IT managers must ensure that databases are integrated to allow for efficient resource use and coherent data analysis.
- Senior management's support is vital for successful data warehouse architecture and integration.
- Common dimensions (e.g., date, product, store) must be shared across different data marts to enable comprehensive reporting.
- Using shared dimensions allows for combined performance measurements across processes, enhancing overall analytical capabilities.

Note 49

Data Warehouse Bus Architecture

- The **data warehouse bus architecture** provides a framework for integrating various data marts while maintaining consistency.
- It allows for separate teams to implement data marts that share standardized dimensions and facts, ensuring interoperability.
- The bus design enables building the enterprise data warehouse incrementally, focusing on manageable data mart pieces.
- A **data warehouse bus matrix** helps document and communicate the architecture by mapping business processes to common dimensions.
- It highlights which dimensions require attention for conformity while facilitating planning and communication across teams.

Note 50

Conformed Dimensions

- **Conformed dimensions** are critical for the data warehouse bus as they ensure uniformity across data marts.
- They must have consistent keys, attribute names, definitions, and values to facilitate

accurate reporting and analysis.
- Nonconformed dimensions can lead to inconsistencies in reporting and hinder effective data integration.
- It is essential to establish standardized dimensions to prevent isolated data marts from becoming legacy systems.

Note 51

Conformed Dimensions

- **Definition**: Conformed dimensions remain consistent across different fact tables, ensuring valid results when data marts are combined.
- **Example**: The date dimension for sales facts is identical to the date dimension for inventory facts, potentially being the same physical table in the database.
- **Granularity**: Most conformed dimensions are defined at the most granular level:
- Customer dimension: individual customer
- Product dimension: lowest level products tracked
- Date dimension: individual day
- **Roll-up Dimensions**: Sometimes dimensions are needed at a higher level of granularity. For example, a weekly inventory snapshot may differ from daily snapshots.
- **Consistency**: Attributes common to both detailed and rolled-up dimensions must be labeled and valued identically.

Note 52

Dimension Authority

- **Role**: The dimension authority is responsible for defining, maintaining, and publishing conformed dimensions.
- **Importance**: Conformed dimensions must be utilized by data mart teams to ensure the enterprise data warehouse functions effectively.
- **Political Challenges**: Securing agreement on conformed dimensions can face more political than technical challenges.
- **Support Required**: Support from top management is crucial for the adoption of conformed dimensions across the organization.
- **Implementation Coordination**: Changes to attributes must be communicated to all data mart teams to maintain consistency.

Note 53

Conformed Facts

- **Definition**: Conformed facts must have the same definitions and equations across multiple data marts, ensuring consistent performance metrics.
- **Key Performance Indicators**: Common KPIs include revenue, profit, and measures of customer satisfaction.
- **Naming Practices**: If a fact cannot be conformed, it should be given a different name to avoid confusion in calculations.

- **Unit of Measure**: Different units of measure for the same fact require careful handling to avoid errors in reports.
- **Multiple Units**: The ideal solution is to carry facts in both units of measure for ease of reporting.

Note 54

Procurement Process Overview

- **Importance**: Procurement is critical for organizations that purchase products or services, influencing cost savings and efficiency.
- **Analytic Requirements**: Common questions include:
- What products are purchased most frequently?
- Are employees adhering to preferred vendor agreements?
- How well are vendors performing?
- **Process Modeling**: The modeling involves understanding various procurement transactions: purchase requisitions, orders, receipts, and payments.
- **Dimension Types**: Key dimensions include product, vendor, and procurement transaction type, with facts including transaction amount and quantity.
- **Design Decisions**: Choices between single versus multiple procurement fact tables depend on user analysis needs and the uniqueness of business processes.

Note 55

Multiple Fact Tables for Procurement Processes

- **Fact Tables**: Several fact tables are established for procurement processes, including:
- **Purchase Requisition Fact**: Contains keys related to requisitions, including:
- Requisition Date Key (FK)
- Product Key (FK)
- Purchase Requisition Dollar Amount
- **Purchase Order Fact**: Related to purchase orders with keys such as:
- Purchase Order Date Key (FK)
- Purchase Order Quantity
- Purchase Order Dollar Amount
- **Shipping Notices Fact**: Captures shipping details, including:
- Shipping Notification Number (DD)
- Shipped Quantity
- **Warehouse Receipts Fact**: Tracks received goods with keys like:
- Warehouse Receipt Date Key (FK)
- Received Quantity
- **Vendor Payment Fact**: Records payment details with:
- Vendor Gross Payment Dollar Amount
- Vendor Net Payment Dollar Amount
- **Dimensions**: Key dimensions include:
- Product Dimension
- Date Dimension
- Vendor Dimension
- Employee Dimension

- Discount Taken Dimension
- Each fact table is linked to various dimensions that provide context for the data recorded.

Note 56

Slowly Changing Dimensions (SCD)

- **Definition**: Slowly Changing Dimensions refer to dimension attributes that change over time, requiring careful management to maintain historical accuracy.
- **Importance of Change Tracking**: Business representatives must be consulted to determine how changes should be managed, as dimension changes can significantly affect data analysis.
- **Types of Slowly Changing Dimensions**:
- **Type 1: Overwrite the Value**
- Simply updates the existing dimension value.
- **Pros**: Fast and easy to implement.
- **Cons**: Loses all historical data.
- **Example**: Changing the department of a product without keeping past records.
- **Type 2: Add a Dimension Row**
- Creates a new row in the dimension table for each change.
- **Pros**: Retains full history of changes.
- **Cons**: Increases dimension table size.
- **Example**: Maintaining separate rows for a product as it moves between departments.
- **Type 3: Add a Dimension Column**
- Adds a new column to capture the previous value, allowing for both old and new values to coexist.
- **Pros**: Allows analysis from both perspectives.
- **Cons**: Limited to tracking one previous value.
- **Example**: Keeping both current and prior department values for analysis.
- **Hybrid Techniques**: Hybrid approaches can be developed to combine these methods, allowing for flexible yet complex data management, often used in scenarios like sales organization realignments.

Note 57

Dimension Table with Multiple Version Overlays

- The dimension table can have **multiple versions** of attributes, allowing users to analyze data based on different historical district assignments for sales reps.
- Each row in the sales rep dimension contains prior district assignments (up to **five versions**).
- The most recent district is labeled as "**Current District**" and will be the primary attribute used in reporting.
- When district boundaries change, a new column (e.g., **District 2002**) is added while the current district is updated to reflect the latest assignments.
- Historical values prior to a sales rep's hiring may be marked as "**Not Applicable**" for years without assignments (e.g., for 1998 and 1999).

Note 58

Unpredictable Changes with Single-Version Overlay

- This approach is used to maintain **historical accuracy** for unpredictable attribute changes while enabling reporting with current values.
- For a product dimension (e.g., IntelliKidz), two department attributes are included: **current department** and **historical department**.
- When departments change, a new row is created (using a **Type 2** response) to capture the change, while the current department is updated across previous entries.
- This method allows for segmentation of historical data and ensures accurate reporting based on current assignments while keeping historical integrity.

Note 59

Hybrid Approach to Changing Dimensions

- The hybrid approach combines different techniques to track changes effectively: **Type 2** for new rows, **Type 3** for current columns, and **Type 1** for overwriting previous entries.
- This combination (referred to as **Type 6**) allows for accurate historical tracking while supporting current reporting needs.
- The balance between flexibility and complexity is crucial; excessive complexity may hinder user experience and understanding.

Note 60

Handling Rapidly Changing Dimensions

- When dimension attributes change frequently (e.g., monthly), traditional slowly changing dimension techniques may not suffice.
- A recommended approach is to separate rapidly changing attributes into distinct dimensions, linking them to the fact table via multiple foreign keys.
- This strategy helps manage data effectively without complicating the main dimension tables.

Note 61

Summary of Procurement Data Management

- The chapter discussed various techniques for managing procurement data and changes to dimension table attributes, highlighting:
- **Type 1**: Overwriting values.
- **Type 2**: Adding new rows for changes.
- **Type 3**: Adding columns for current values.
- Hybrid techniques that combine these basic strategies were also explored, emphasizing the importance of balancing complexity with user needs.

Note 62

Order Management Overview

- Order management consists of critical processes including **order processing**, **shipment**, and **invoicing**, which are vital for tracking sales metrics.
- The chapter will cover:
- Various order management transactions.
- **Fact table normalization** considerations.
- The role of **date dimensions**.
- Handling different granularity of facts (header vs line items).
- **Accumulating snapshot** schema for analyzing order fulfillment.

Note 63

Transaction and Fact Table Normalization

- Order transaction fact tables typically consist of one row per **line item**, including key metrics such as order quantity and net dollar amounts.
- Fact normalization is sometimes considered to reduce redundancy, but excessive normalization can complicate the schema and increase row counts significantly.
- It is generally preferred to keep related facts in the same row for easier calculations and clearer reporting.

Note 64

Role-Playing Date Dimensions

- Each transaction may involve multiple dates (e.g., order date and requested ship date), necessitating unique foreign keys for each in the fact table.
- Role-playing is achieved by creating views to simulate independent date dimensions while maintaining a single underlying date dimension table.
- Proper labeling in views is essential to avoid confusion in reports, ensuring clarity in data analysis.

Note 65

Product Dimension Characteristics

- The product dimension table typically contains numerous descriptive columns, often exceeding **100 attributes**.
- Products may belong to multiple hierarchies, and all hierarchical data should be presented in a **flattened, denormalized structure** to facilitate browsing.
- Operational product keys must be remapped to **surrogate keys** to avoid complications from duplicates and changes over time.

Note 66

Customer Ship-To Dimension Hierarchies

- **Multiple Hierarchies**: Customer ship-to dimensions can have several independent hierarchies, such as:
- **Geographic Hierarchy**: Defined by ship-to location, typically structured as:
- City
- County
- State
- U.S. ZIP code (with first digit indicating region)
- **Organizational Hierarchy**: If the customer is a corporation, there are bill-to and corporation relationships for each ship-to.
- **Complex Relationships**: It's common for multiple ship-tos to associate with one bill-to, leading to:
- A many-to-one relationship that may require adjustments in the dimension design.
- Two rows for each unique ship-to/bill-to combination if exceptions are frequent.
- **Design Implications**: Designers may choose to treat ship-to and bill-to as separate dimensions linked via fact tables if many-to-many relationships exist.
- **Recursive Relationships**: Further exploration of customer organizational hierarchies will occur in Chapter 6.

Note 67

Sales Organization Attributes

- **Integration Considerations**: Designers must decide whether to:
- **Combine Sales Org Attributes** with the customer ship-to dimension if there's a strong correlation (one-to-one or many-to-one relationship).
- Keep separate dimensions if the relationship is complex or many-to-many, especially if it varies over time or with other dimensions (e.g., product).
- **Factless Fact Tables**: Users may want to analyze historical assignments of sales reps to customers, resulting in:
- Creating a factless fact table to capture sales rep coverage over time, including effective and expiration dates for assignments.

Note 68

Deal Dimension

- **Definition**: The deal dimension outlines incentives offered to customers that potentially influence purchase decisions, also referred to as contracts.
- **Correlation of Attributes**: Similar to promotion dimensions, if deal components (terms, allowances, incentives) are correlated, they should be combined into a single deal dimension. If uncorrelated, separate dimensions may be appropriate to avoid complexity.
- **Design Considerations**: In large fact tables, keeping the deal dimension as a single entity can simplify management. A deal dimension with less than 100,000 rows is manageable.

Note 69

Degenerate Dimensions

- **Definition**: Each line item in the orders fact table includes an order number as a degenerate dimension, not tied to an order header table, useful for grouping line items.
- **Operational Use**: Degenerate dimensions should not just serve as placeholders; they can provide valuable identifiers for transactional data.

Note 70

Junk Dimensions

- **Purpose**: Junk dimensions collect low-cardinality flags and indicators that do not belong to other dimensions, preventing fact table bloat.
- **Examples**: An example of a junk dimension can include multiple two-value indicators, e.g., payment types, which are consolidated into a single dimension for efficiency.
- **Design Strategy**: When dealing with flags, if the number of unique combinations is high, separate dimensions may be warranted. Otherwise, a junk dimension can be an effective solution.

Note 71

Handling Multiple Currencies

- **Fact Table Design**: For multinational transactions, each order fact should include:
- Local currency amount
- Equivalent amount in a standard corporate currency (e.g., U.S. dollars).
- **Currency Conversion Table**: An additional fact table may be needed to manage currency conversions, allowing queries in various currencies.
- **Reporting Flexibility**: This approach supports reporting in multiple currencies, accommodating diverse managerial needs across different regions.

Note 72

Header and Line Item Facts

- **Granularity Issues**: There may be discrepancies in the granularity of facts (e.g., shipping charges at order level vs. product level).
- **Allocation**: Allocating header-level facts to line-item levels enables comprehensive analysis across dimensions, although this can lead to organizational challenges in agreement on allocation rules.

Note 73

Overall Considerations in Dimensional Design

- **Complexity Management**: Designers should aim for a manageable number of dimensions, considering the need to avoid excessive complexity by combining similar dimensions where feasible.
- **Fact Table Efficiency**: The fact table should be streamlined, focusing on clarity and efficiency in representing relationships between dimensions without unnecessary duplication.

Note 74

Shipping Charges and Fact Allocation

- If shipping charges and header-level facts cannot be allocated successfully, they should be presented in an aggregate table for the overall order.
- Allocating facts to a lower level is preferred due to usability issues with higher-level fact tables.
- Without allocation, header facts cannot be explored by product, as the product isn't identified in a header-grain fact table.
- Mixing fact granularities (e.g., order and order line facts) within a single fact table should be avoided.
- Ideally, a finance or business team should lead the allocation effort, not the data warehouse team.

Note 75

Invoice Transactions

- Invoicing typically occurs when products are shipped, creating an invoice with multiple line items for each product shipped.
- Each line item includes various prices, discounts, and allowances, with an extended net amount available.
- At the time of shipment, the business knows about the list prices and manufacturing/ distribution costs, providing insights into the business state.
- The shipment invoice fact table includes:
- Products
- Customers
- Contracts and deals
- Discounts and allowances
- Revenue generated
- Variable and fixed costs associated with manufacturing and delivery
- Customer satisfaction metrics, such as on-time shipment.

Note 76

Profit and Loss Facts

- Organizations with robust ERP systems can identify revenues and costs associated with shipping products.

- The P&L statement outlines revenues and costs, ranging from the undiscounted value of shipped products to the contribution remaining after discounts and costs.
- Key elements of the P&L statement include:
- Quantity shipped
- Extended gross invoice amount (quantity shipped x list unit price)
- Extended allowance amount for deal-related allowances
- Extended discount amount for volume or payment-term discounts
- Extended net invoice amount (gross amount less allowances and discounts)
- Extended fixed and variable manufacturing costs
- Contribution amount (net invoice less all costs).
- Internal cost amounts are not typically shown to customers.

Note 77

Customer Satisfaction Metrics

- In addition to P&L facts, businesses often track customer satisfaction metrics related to line items.
- Satisfaction metrics can be added to the fact table with additive flags (1 for yes, 0 for no).
- A customer satisfaction dimension can be created to summarize these metrics into text equivalents for reporting.

Note 78

Accumulating Snapshot for Order Fulfillment

- The order management process is visualized as a pipeline, with unique transactions at each stage.
- An accumulating snapshot fact table provides a holistic view of order fulfillment, showing product movement through the pipeline.
- It captures multiple dates representing major milestones, and rows are updated as new information becomes available.
- The grain is one row per order line item, modified as the order progresses.
- Useful for uniquely identified products, the accumulating snapshot helps understand throughput and efficiency, identifying bottlenecks.

Note 79

Fact Table Design and Conversion Factors

- Storing **conversion factors** in the **fact table** instead of the dimension table is recommended to avoid miscalculations.
- Example: In an orders pipeline fact table, instead of having 50 facts (10 basic quantities with 5 units), store 10 quantities and 4 conversion factors, resulting in 14 total facts.
- This design allows performance analysis across the **value chain** using different units of measure.
- The fact table is delivered to business users through views, simplifying the user interface.
- Intra-row computations are efficient, and the extra computation for conversion is negligible

compared to DBMS overhead.
**

Note 80

Benefits of Storing Conversion Factors in Fact Tables

- Storing conversion factors in the fact table ensures they are used correctly in calculations.
- Reduces pressure on the product dimension table to frequently update for minor factor changes.
- Factors evolve like facts, making them more suitable for fact tables than dimension attributes.

Note 81

Historical vs. Predictive Metrics

- Discussions often focus on **historical metrics** (rear-view mirror metrics) to analyze past product performance.
- Organizations are increasingly interested in **predictive metrics** to anticipate future order activity based on drivers like prospecting or quoting.
- CRM solutions are implemented to enhance understanding of leading indicators, despite challenges with legacy data sources.
- Accurate capture of early indicators can be added to the data warehouse once established.

Note 82

Types of Fact Tables

- There are three fundamental types of fact tables:
- **Transaction Fact Tables**: Represent events at a point in time; one row per transaction.
- **Periodic Snapshot Fact Tables**: Capture cumulative performance at regular intervals; represents aggregated data.
- **Accumulating Snapshot Fact Tables**: Track the status of transactions over time, updated regularly.
- Each type serves a different purpose and is essential for a complete business perspective.

Note 83

Transaction Fact Tables

- Transaction fact tables provide detailed insight into operations at the transaction level, allowing for granular analyses.
- They are linked to multiple rows for customers/products due to repeated transactions, supporting detailed behavioral analysis.

- However, transaction-level data alone cannot answer all business questions, necessitating other fact types.

Note 84

Periodic Snapshot Fact Tables

- Periodic snapshots provide a trendable view of business performance at regular intervals (daily, weekly, monthly).
- They aggregate transaction data to present cumulative performance, essential for reporting metrics.
- Snapshot tables often have fewer dimensions than transaction tables but more facts, reflecting the overall performance over time.

Note 85

Accumulating Snapshot Fact Tables

- Accumulating snapshots represent ongoing processes, such as orders and shipments, with records updated as activity occurs.
- They typically feature multiple date stamps to track progress through various phases of a transaction.
- The design can work in conjunction with periodic snapshots for comprehensive reporting.

Note 86

Real-Time Partitions in Data Warehousing

- New requirements necessitate real-time data integration alongside historical data in data warehouses.
- Real-time partitions must link seamlessly with static data warehouse tables, containing all activities since the last update.
- These partitions are lightly indexed to allow continuous data updates and provide fast query responses.
- Different structures exist for real-time partitions depending on the fact table type (transaction, periodic, accumulating).

Note 87

Summary of Order Management Process

- The chapter discussed various aspects of order management, including dimension role-playing, equivalent units of measure, and challenges in modeling order data.
- Key concepts included accumulating snapshot fact tables to track product/order status in the order fulfillment pipeline.
- A summary of the differences between the three fundamental fact table types was

provided, along with suggestions for handling near real-time reporting.

Note 88

Minidimensions and Customer Data Management

- **Minidimensions** are used for attribute browsing and change tracking in large dimensions, especially when dealing with variable-width attribute sets.
- The **Type 2 Slowly Changing Dimension (SCD)** technique affects dimension counts and should be considered in design.
- **Behavior study groups** help track customers with common characteristics or behaviors.
- Considerations for **commercial customer hierarchies** include both fixed and variable depths.
- The integration of customer data from various sources is crucial for comprehensive analysis.

Note 89

CRM Overview

- Organizations are shifting from a **product-centric** to a **customer-centric** approach, emphasizing the importance of understanding customer needs.
- **CRM (Customer Relationship Management)** aims to enhance relationships with customers, maximizing their lifetime value.
- Key benefits of CRM include:
- Increased revenue and operational efficiencies
- Improved sales effectiveness and closure rates
- Enhanced customer satisfaction and retention
- Organizations often have varied views of customers based on product lines and functions, requiring a unified approach to data.
- Successful CRM implementation demands a strategic plan and organizational commitment to integrate people, processes, and technology.

Note 90

Operational and Analytic CRM

- CRM addresses both **operational** and **analytic** requirements, focusing on data collection at every customer interaction.
- Operational CRM synchronizes **customer-facing processes** across sales, marketing, operations, and service.
- The **data warehouse** is fundamental for storing and analyzing historical customer interaction metrics, providing a 360-degree customer view.
- **Analytic CRM** uses integrated customer data to optimize future interactions and identify up-sell and cross-sell opportunities.
- Feedback loops from analytic CRM can enhance operational systems, ensuring timely responses to customer interactions.

Note 91

Data Warehouse and CRM Integration

- The data warehouse must evolve to accommodate the growing customer information from CRM initiatives, emphasizing **data integration** and **conformed dimensions**.
- Organizations face a **buy vs. build** decision regarding CRM solutions, balancing immediate implementation against tailored long-term needs.
- Packaged CRM solutions should not operate as isolated data islands but must integrate with the existing data warehouse to avoid inconsistencies.
- A well-maintained **conformed customer dimension** is essential for effective analysis, often requiring significant design and maintenance efforts.

Note 92

Customer Dimension Design Considerations

- The customer dimension is challenging due to its potential size and complexity, necessitating careful design to accommodate a vast number of attributes.
- **Name and address parsing** are critical; simplistic data structures lead to data quality issues. Consider a detailed attribute breakdown for better segmentation and understanding.
- An example of a well-structured customer dimension includes:
- Multiple elements for names (e.g., salutation, first name, surname).
- Comprehensive address attributes to support accurate communication.
- **International considerations** for names and addresses require consistent representation and cultural correctness, accommodating regional differences in addressing and telephone formats.

Note 93

Common Customer Attributes and Segmentation

- Dates are frequently included in customer dimensions (e.g., first purchase date), often linked to a **date dimension** for enhanced analysis.
- **Segmentation attributes** like gender, ethnicity, and income are valuable for understanding customer profiles and behaviors.
- The more detailed and descriptive the customer attributes, the more robust and insightful the analysis will be, supporting better decision-making and targeted marketing strategies.

Note 94

Customer Dimension Attributes

- **Recency, Frequency, Intensity**: These metrics include:
- **Recency**: Date of last purchase.
- **Frequency**: Total purchase transaction count.

- **Intensity**: Total net purchase amount.
- Cluster labels from data mining of these measures.
- **Business-specific segments**: Identifiers for preferred customers.
- **Customer scores**: Characteristics include:
- Purchase behavior.
- Payment behavior.
- Product preferences.
- Propensity to churn and probability of default.
- **Aggregated Facts**: Helps in filtering customers based on spending metrics, which can be stored as attributes for ease of access.
- **Maintenance of Attributes**: Focus on frequently used attributes to minimize updates; descriptive values like "High Spender" are preferred for clarity and consistency.

Note 95

Aggregated Facts as Attributes

- Users can filter customers based on **aggregated performance metrics**, such as:
- Spending over a certain amount last year.
- Lifetime purchases.
- **Advantages**:
- Simplifies queries by allowing constraints on spending attributes directly.
- Enhances usability and performance.
- **Challenges**:
- Requires diligent maintenance to ensure accuracy and consistency with actual data.
- Frequent updates can be burdensome; prefer less frequently changing attributes.

Note 96

Dimension Outriggers and Snowflaking

- **Snowflaking**: Generally discouraged in data warehouse environments due to complexity and performance issues.
- **Dimension Outriggers**: Permissible in specific cases, e.g., when demographic data is consistent across many customers:
- Example: Customer demographics data from external providers.
- Outriggers save space and keep dimensions manageable, but excessive use can complicate the design.

Note 97

Handling Large Customer Dimensions

- **Challenges** with large customer dimensions:
- Browsing performance can deteriorate.
- Tracking changes becomes complex.
- **Minidimension Technique**: Utilizes a separate dimension for frequently changing attributes to improve performance:

- Example: Separate dimension for demographic attributes like age and income.
- **Data Management**: Use banded ranges for continuous variables to reduce combinations and maintain efficiency.

Note 98

Demographic Minidimensions

- **Creating a Demographic Minidimension**:
- Each unique combination of demographic attributes gets one row.
- Continuous attributes like income should be banded to limit combinations.
- **Tracking Changes**: The fact table links to demographic attributes to track changes over time without cluttering the main customer dimension.
- **Implementation**:
- Include demographic keys in both the fact table and the customer dimension for efficient data retrieval.

Note 99

Variable-Width Attribute Set

- **Variable Number of Attributes**: Differences in data for prospects vs. customers can lead to inefficiencies in data storage:
- Prospects have fewer attributes compared to customers.
- **Minidimension Use**: Break down dimensions to manage data efficiently, especially with fixed-width databases:
- Base dimension for common attributes shared by prospects and customers.
- Separate minidimension for customer-specific attributes.
- **Data Management**: Store minimal attributes for prospects while maintaining detailed data for customers to avoid unnecessary storage issues.

Note 100

Implications of Type 2 Changes in Customer Dimensions

- **Type 2 Slowly Changing Dimensions**: Suitable for organizations with fewer customers (under 100,000):
- Allows tracking of historical changes without overcounting.
- **Counting Customers**: Use **COUNT DISTINCT** on unique identifiers to avoid duplication in counts.
- **Most Recent Row Indicator**: Helpful for providing counts based on the latest customer attributes without multiple row issues.

Note 101

Customer Count Analysis

To determine the number of customers at a specific historical point in time, such as the beginning of **2002**:

- **Effective Date Constraint**: Limit effective dates to those on or before **1/1/2002**.
- **Expiration Date Constraint**: Limit expiration dates to those on or after **1/1/2002**.
- This method ensures that only customers valid on a specific date are counted.
- Comparison operators used depend on the business rules governing effective and expiration dates.
- Alternatively, surrogate date keys can be joined to a date dimension for more complex queries.

Note 102

Customer Behavior Study Groups

Customer analysis can evolve from simple queries to complex inquiries:

- **Simple Query Example**: Total sales to customers in a geographic area over the past year.
- **Complex Query Example**: Identifying customers whose purchases exceeded their average from last year in the past month.
- Data access tools may allow for **embedded subqueries** or **multipass SQL capabilities** to handle complex requests.
- To create behavior study groups, capture customer keys from specific queries (e.g., top 100 customers or those who spent over $1,000 last month) and store them as a dimension table for further analysis.
- This avoids reprocessing and allows for focused analysis on specific customer groups.

Note 103

Joining Behavior Study Groups

The behavior study group dimension is linked to the customer dimension:

- **Join Type**: Equijoin to the natural key (e.g., Customer ID).
- The resulting dimensional model can be simplified using views to hide complex joins.
- Ensure the behavior study group dimension is clearly labeled for identification (e.g., for top 100 customers).
- Some data tools can analyze this simplified schema without performance penalties.
- This approach requires an interface for creating and managing behavior study group tables, which impacts DBA responsibilities.

Note 104

Commercial Customer Hierarchies

Modeling customer hierarchies can be complex due to nested relationships:

- **Fixed-Depth Hierarchies**: Suitable for predictable structures with a limited number of levels (e.g., corporate parent, business unit, regional office).
- Populate levels accordingly, even duplicating values for simpler structures.
- **Variable-Depth Hierarchies**: More complex but allows for arbitrary structures.
- Use a recursive parent key pointer, though this complicates SQL usage.

- Alternatively, a bridge table can be used for navigating hierarchies, facilitating standard SQL queries.

Note 105

Bridge Tables for Hierarchies

Using bridge tables can effectively manage organizational hierarchies:
- **Structure**: The bridge table links customer dimensions to fact tables, allowing analysis across hierarchical levels.
- **Pathways**: Each row in the bridge table represents a pathway from a customer entity to its subsidiaries.
- **Hierarchical Navigation**: Joins allow descending or ascending through the hierarchy, with careful management to avoid double counting in fact tables.
- **Administrative Considerations**: Maintaining bridge tables requires tracking organizational changes and may involve implementing effective/expiration dates.

Note 106

Combining Customer Data Sources

Populating the customer dimension involves integrating multiple data sources:
- **Challenges**: Unique customers may have multiple identifiers across systems, complicating consolidation efforts.
- **Matching Logic**: Implement customer matching and deduplication to ensure data integrity and accuracy.
- **Address Validation**: Use algorithms and lookup directories for address validation, essential for accurate customer information.
- **External Matching Services**: Consider investing in software that provides robust matching capabilities to streamline data consolidation.

Note 107

Analyzing Customer Data from Multiple Business Processes

- Data warehouses should be built **process by process**, focusing on **conformed dimensions** for cross-process integration.
- Example of querying sales and support service fact tables to analyze a customer's purchase and service history.
- **Customer foreign keys** are essential for joining fact tables; however, many-to-one-to-many joins may yield incorrect results due to differences in cardinality.
- **Example:** A fact table for customer solicitations and another for responses indicates a one-to-many relationship; simultaneous joining may not yield accurate data.
- The **multipass SQL approach** is recommended to manage queries effectively across different fact tables and improve performance parameters.

Note 108

Customer Dimension Design Issues

- Focus on the **customer dimension** in CRM, emphasizing the importance of accurate parsing of names and addresses.
- Discussed attributes in customer dimensions: segmentation, dates, and aggregated facts.
- **Minidimensions** can be helpful for frequently analyzed attributes, especially when the customer dimension has millions of rows.
- Consideration for **behavioral study group dimensions** to reflect shared traits among customers.
- **Customer hierarchies** often require bridge tables for effective querying, particularly with variable-depth structures.

Note 109

General Ledger Overview

- The **general ledger (G/L)** is a core financial system, integrating data from various subledgers such as accounts payable and receivable.
- Focus on **periodic snapshots** and transactions for accurate financial reporting.
- The **chart of accounts** serves as an intelligent key for identifying accounts, types, and organizational structures.
- Period close processes require careful reconciliation and finalization of financial results before reporting.
- Emphasis on creating a data warehouse as a single source of accurate financial information for diverse organizational users.

Note 110

General Ledger Data Structure

- The **G/L periodic snapshot** captures account balances at the end of each fiscal period, providing foundational financial data.
- Designers need to be cautious about storing **to-date totals** in fact tables as they may not align with the grain of the data.
- G/L data often involves **multiple currencies**, requiring separate representations for local and corporate currencies.
- A detailed **journal entry transaction schema** is necessary for users needing granular transaction details behind the periodic snapshots.
- The **debit/credit indicator** in journal entries can be optimized for storage efficiency, possibly using a two-row decode dimension table.

Note 111

Multiple Fiscal Accounting Calendars

- Organizations may operate under **multiple fiscal accounting calendars**, requiring careful data structuring in the data warehouse.

- Daily date dimensions may need to align with both standard calendar and unique fiscal periods, especially in complex cases.
- For **subsidiaries with unique fiscal calendars**, a date dimension outrigger can be employed for effective filtering and organization.
- Each row in the daily date dimension may need to reflect multiple fiscal attributes, enhancing the ability to analyze data across various accounting periods.
- Consideration of fiscal accounting alignment is crucial for accurate financial reporting and analysis.